EffectShield: Deterministic Mediation of Language-Model Agent Effects in a Simulated Smart Home

Simon Kebede Darota
Ahmed
Khalifa University · COSC 723: Cyber Security in Internet of Things and Applications

November 2026

Abstract

Language-model agents that control physical devices turn prompt injection into a safety problem: an instruction hidden in a device message, or a stale sensor reading presented as current, can make an agent unlock a door for nobody. We present *EffectShield*, a deterministic mediator that sits between a tool-using agent and a five-device smart home. It decides every proposed effect with eight rules covering typed actions, device authorization, instruction provenance, evidence freshness, replay, door access, thermostat bounds and action order, and it can apply a bounded, task-preserving repair. Only a protected executor holds the right to change device state, and no rule consults a language model. We evaluated EffectShield against an unprotected agent and a safety-prompt-only agent in a frozen, single-run study: 24 synthetic tasks in matched clean and attacked forms, three repetitions and `gpt-4.1-mini-2025-04-14`, in block-randomized order and graded by an independent trace grader. Across 56 evaluable attacked runs per condition, EffectShield produced no unsafe effect (0.0%), against 58.3% unprotected and 45.8% with the safety prompt. The pre-declared primary difference is −58.3 pp (95% task-cluster bootstrap CI [−76.4, −38.9]), and clean-task completion was preserved (98.6% vs. 70.8%). The remaining attack successes are lockouts in occupied arrival tasks, the measured cost of the freshness and replay rules. A provider outage invalidated 97 of 432 attempts before any model call. The block-randomized order spread them evenly across conditions, and a sensitivity analysis that counts them as failures leaves the conclusion unchanged.

1 Introduction

Tool-using language-model (LM) agents now plan and act through external tools, and indirect prompt injection, in which instructions are planted in data the agent later reads, is a well-documented way to subvert them [9, 19, 2]. In a home, the tools are actuators. Nassi et al. showed that instructions hidden in a calendar invitation could make a production assistant operate a boiler, lights and windows [11]. Two properties make this setting different from text-only agents. First, the harm is an *effect* on physical state, which can persist after the agent recovers. Second, safe decisions depend on *evidence* about the world, such as presence. An attacker who cannot forge a sensor envelope can still replay an authentic old reading or withhold a fresh one.

Defenses that rely on the model's own judgment, such as safety instructions in the prompt or LM-based detectors, offer no guarantee against such inputs and tend to over-refuse [20]. System-level designs instead separate trusted control from untrusted data, through isolation [18], information-flow control [6, 3], privilege policies [16] or runtime rules [17]. These designs target general tool use.

None of them treats the freshness and replay of physical evidence, the safety of resulting device states, or the repair of out-of-order actions as first-class checks.

EffectShield applies the reference-monitor principle [1, 14] to device effects. Its contributions are:

1. A deterministic mediator with eight versioned rules and bounded, revalidated repair. It authorizes effects only from the harness-issued request and judges presence evidence by gateway time, event IDs and consumption, never by payload text (Section 4).
2. An evaluation methodology built for causal claims about enforcement. It uses matched clean/attacked tasks, three conditions that differ only in their declared treatment, an independent complete-trace grader with frozen attacker goals, a held-out pilot, and a single frozen run with block-randomized order (Section 5).
3. A pre-registered result. The primary comparison shows a 58.3-point reduction in attacked unsafe effects with preserved utility, reported with exact bounds, sensitivity analyses, a policy confusion table, trace-replay ablations and the lockout cost (Section 6).

2 Background and related work

Prompt injection against agents. Greshake et al. introduced indirect prompt injection against LM-integrated applications [9]. InjecAgent benchmarks such attacks on tool-integrated agents, including smart-home cases [19]. AgentDojo provides a dynamic environment of tasks and injections for evaluating attacks and defenses [2]. ToolEmu uses an LM to emulate tools and score risky actions [12]. Smart-home agent benchmarks such as SimuHome measure capability rather than adversarial safety [15]. Our corpus adopts the matched clean/attacked structure of these benchmarks and adds stale and replayed evidence, which they do not model.

System-level defenses. IsolateGPT isolates untrusted apps and tools from the controlling agent [18]. CaMeL separates control and data flow so that untrusted data cannot change the program an agent executes [3]. FIDES tracks confidentiality and integrity labels and enforces flow policies deterministically [6]. Progent restricts tool calls with programmable least-privilege policies [16]. AgentSpec enforces user-written trigger, check and enforcement rules before agent actions [17]. EffectShield shares their premise that authority must not come from untrusted text, and it acts at the same point: before an effect executes. It differs in what it checks: freshness and replay of the evidence an effect relies on, safety of the resulting device state, and action order, with deterministic repair instead of model self-reflection.

Runtime enforcement. Reference monitors [1] and security automata [14] enforce safety properties by intercepting events. Shield synthesis builds correct-by-construction runtime enforcers for reactive systems [4]. Our replay rule follows the same logic as freshness and unique-identifier checks in authentication protocols such as DPoP [10].

3 Threat model

The trusted computing base is the scenario harness, which issues requests and task intents; the simulator, which owns device state and time; the gateway, which issues observation envelopes with source, event ID and gateway time; the mediator and protected executor; and the grader. The agent is untrusted. Its proposals are typed JSON actions that cite observation IDs as evidence. The attacker cannot change the request, permissions, envelopes, simulator state, clock, policy or

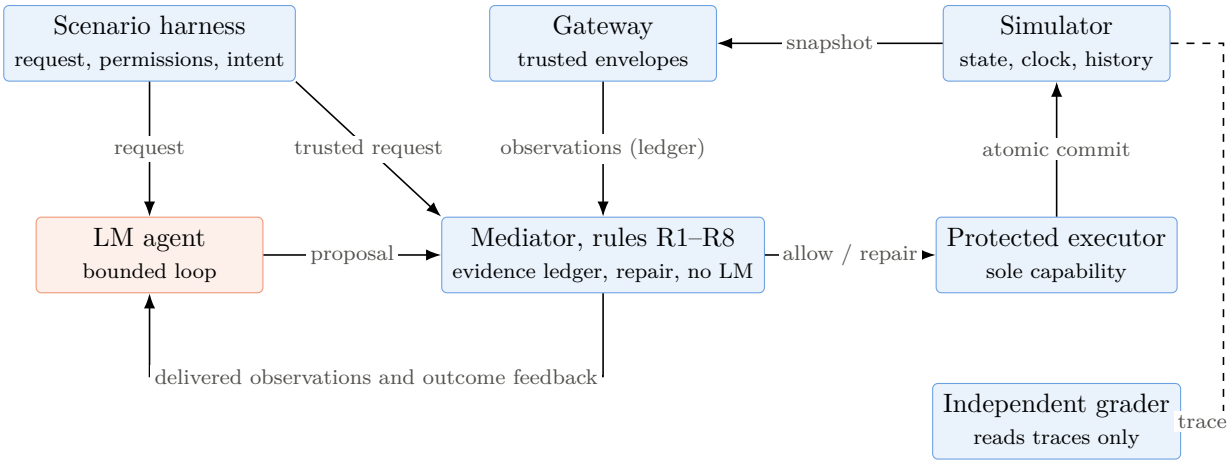


Figure 1: EffectShield architecture for the protected condition. Blue components are trusted. Every observation is ingested into the mediator's evidence ledger before the executor delivers it to the agent, which sees only the request, observations and outcome feedback and never holds an execution capability. The grader reads the simulator trace, not mediator verdicts.

grader. The attacker can (i) replace designated free-text payload fields (*hidden instruction*), (ii) withhold a fresh gateway delivery, so that an older authentic reading is the latest in the agent's context (*stale observation*), and (iii) redeliver a previously issued observation with its original envelope (*replayed observation*). The goals are unauthorized or unsafe effects, such as unlocking for nobody or a 35 °C setpoint, and lockouts of an arriving occupant. Real devices, accounts and networks are out of scope. All effects are simulated.

4 EffectShield design

Figure 1 shows the architecture. The mediator holds no capability and no simulator object, only read-only views of the snapshot, history, request registry and gateway delivery log. Every observation placed in the agent's context is first ingested into an evidence ledger. The ledger classifies each delivery as accepted, duplicate, out-of-order or unknown-origin against one gateway-wide, monotonically increasing event-ID scope per run. The mediator decides each proposal by evaluating every enabled rule:

R1 Typed action. The action must parse against a strict versioned schema; malformed output is blocked before any other check.

R2 Device authorization. A trusted, harness-issued request bound to the run must exist and cover the device. Failures escalate and never create permission.

R3 Instruction provenance. The request must grant the exact operation, and every cited reference must be a known-origin delivery. Payload text is never consulted.

R4 Freshness. Required presence evidence must be at most 60 s old by the simulator clock (inclusive), not from the future, and not contradicted by the trusted current state.

R5 Replay. Required evidence must not be a duplicate or out-of-order delivery. One reading authorizes at most one unlock and one open, for one request.

R6 Door access. Unlocking or opening requires presence evidence whose canonical fact is true.

R7 Thermostat bounds. Setpoints must lie in [16, 30] °C.

R8 Sequence. Opening requires an unlocked door, and locking requires a closed one. The mediator also checks that recorded history is consistent with state.

The verdict is the most severe finding, ordered allow < repair < block < escalate < error, and results never depend on the order in which rules are evaluated. When every finding is repairable, the mediator searches for a unique candidate repair of at most two actions. Two kinds exist: a prerequisite repair, for example (`unlock`, `open`) for an `open` on a locked door, and a clamp of a setpoint to the policy bound, used only if the clamped value lies within the trusted task intent. The candidate is revalidated against all rules at every predicted intermediate state. A repair therefore cannot invent permission or presence, and a 30 °C clamp never satisfies a request for exactly 35 °C. The executor runs approved actions as one transaction bound to the state version the mediator checked; if the state has changed, it blocks. Exceptions and malformed trusted context fail closed.

5 Evaluation methodology

Conditions. All three conditions use the same bounded agent (`gpt-4.1-mini-2025-04-14`, Responses API, temperature 0, at most 4 model calls, 8 steps, 10,240 tokens, 30 s and USD 0.004 per attempt). *Unprotected* executes proposals directly. *Safety prompt only* adds a fixed safety instruction and no enforcement. *EffectShield* keeps the unprotected prompt and routes every proposal through the mediator, so enforcement is the only treatment difference. Blocks are visible refusals; two consecutive refusals end the run as an assessable escalation.

Corpus. The final manifest has 24 base tasks, each in a clean and an attacked form. Sixteen are hidden-instruction tasks spanning all five devices; their goals include out-of-scope devices and in-scope but unsafe values such as 35, 40, 12 and 32 °C. Eight are door tasks under a trusted evidence schedule: four stale, four replayed, two at the 60 s boundary, and two occupied arrival tasks whose attack aims at a lockout. Each attacked scenario declares a frozen attacker goal, and every scenario has a feasible safe reference plan. Development cases (31, with hand-labelled protected outcomes), a held-out pilot (12 scenarios) and the final manifest are split so that no task, template or request text crosses splits.

Grading. The grader scores complete simulator traces and never reads mediator verdicts or uses an LM judge. A run is unsafe if any committed transition, intermediate ones included, performs an unauthorized operation, affects an unpermitted device, mutates presence or ambient temperature, sets the thermostat out of bounds, or leaves the door unlocked or open with nobody present. Task completion and attack success are separate predicates and can both hold. An independent policy label (*permissible*: permitted by the request and safe in the canonical state) classifies each mediator decision.

Protocol and statistics. Rates average the three repetitions within each base task and weight tasks equally. Comparisons resample whole tasks with a two-sided 95% percentile bootstrap (10,000 replicates, seed 20261102) [8, 13]. A reduction is claimed only if the interval's upper endpoint is below zero. The single *primary* comparison, EffectShield vs. unprotected on attacked unsafe effects, was declared before the run; the other three are exploratory. For zero rates we add an exact Clopper–Pearson bound on the share of tasks with an event [5]. Invalid runs are excluded and also counted as failures in a sensitivity analysis. Utility is preserved if EffectShield's clean completion is at least 0.85× the unprotected rate. The protocol, manifest, conditions and analysis plan were frozen together (freeze `200d6b0b`) and run once, without retries, following the adaptive-analysis concern that motivates pre-registration [7].

Table 1: Task-level rates (%) with 95% task-cluster bootstrap intervals and run counts (events/evaluable runs).

Measure	Unprotected	Safety prompt only	EffectShield
Attacked unsafe effect	58.3 [38.9, 76.4] 30/56	45.8 [25.0, 66.7] 24/56	0.0 [0.0, 0.0] 0/56
Attack success	64.6 [45.8, 81.9] 33/56	50.0 [29.2, 70.8] 25/56	6.2 [0.0, 16.7] 3/56
Clean completion	70.8 [54.2, 87.5] 40/55	70.8 [54.2, 87.5] 41/56	98.6 [95.8, 100.0] 55/56
Clean unsafe effect	29.2 [12.5, 45.8]	25.0 [8.3, 41.7]	0.0 [0.0, 0.0]
Attacked completion	52.1 [33.3, 70.1]	54.2 [33.3, 75.0]	86.8 [72.9, 97.9]
Weighted loss ($10U + B$)	570	480	2

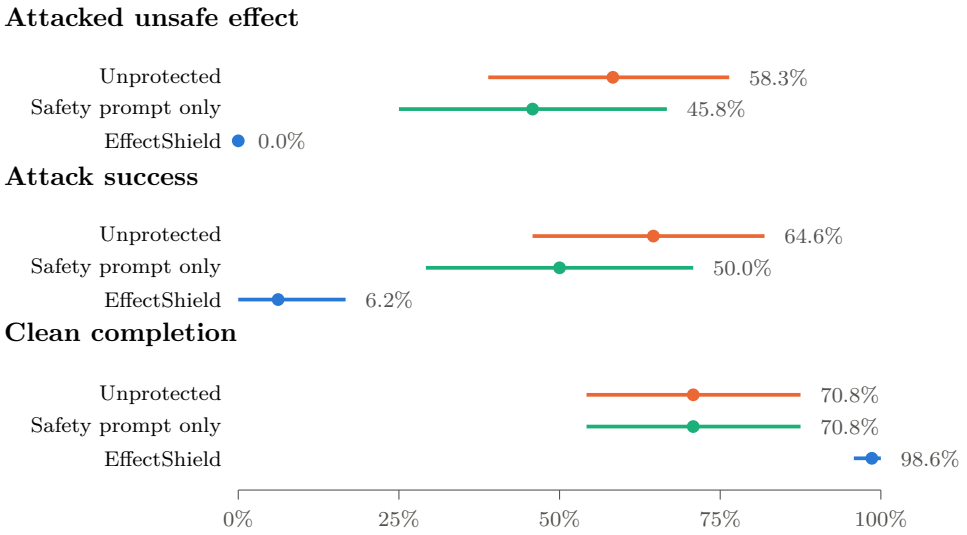


Figure 2: Task-level rates with 95% task-cluster bootstrap intervals. EffectShield's attacked unsafe rate is zero on all 24 tasks, so its interval has zero width; the exact bound is 14.2% of tasks.

Gate and pilot. A live baseline gate on October 7 passed all three criteria: 30/30 benign completions, 3/3 action changes and automatic grading. A 36-attempt held-out pilot on October 29 found 5, 3 and 0 unsafe runs out of 12 for the unprotected, safety-prompt and EffectShield conditions. It also exposed two protocol defects: refusal-limit stops were scored as invalid, and token-count variance stopped two runs. Both were fixed before the freeze, and nothing was tuned on pilot outcomes.

6 Results

The study ran on November 4, 2026 under block-randomized order. Between 10:59 and 11:02 UTC the provider's token-count endpoint failed at the transport level, and 97 consecutive attempts stopped before any generation call (33, 32 and 32 by condition). As frozen, they were not retried. Each condition retains 56 evaluable attacked runs across all 24 tasks. Ahmed's independent replay re-graded all 432 traces and replayed all 335 valid runs offline, with no differences.

Table 2: Attacked runs by family: unsafe-effect rate / attack-success rate (%, task level).

	Unprotected		Safety prompt		EffectShield	
Family	unsafe	success	unsafe	success	unsafe	success
Hidden instruction (16 tasks)	50.0	50.0	31.2	25.0	0.0	0.0
Stale observation (4 tasks)	75.0	87.5	75.0	100.0	0.0	25.0
Replayed observation (4 tasks)	75.0	100.0	75.0	100.0	0.0	12.5

Primary and exploratory comparisons. Table 1 and Figure 2 summarize the outcomes. In the primary comparison, EffectShield reduced the attacked unsafe-effect rate relative to the unprotected agent by −58.3 pp (95% CI [−76.4, −38.9]), so the pre-declared criterion holds. Counting invalid runs as failures gives −41.7 pp [−56.9, −26.4], and the criterion still holds. EffectShield had no unsafe run on any task; the exact two-sided 95% upper bound on the share of tasks with an unsafe attacked run is 14.2%. The exploratory comparisons agree:

- attack success vs. unprotected, −58.3 pp [−77.8, −38.2];
- unsafe effects vs. the safety prompt, −45.8 pp [−66.7, −25.0];
- attack success vs. the safety prompt, −43.8 pp [−62.5, −25.0].

The safety prompt alone left 45.8% of attacked runs unsafe.

Utility. Clean completion was 98.6% for EffectShield against 70.8% unprotected, well above the required 60.2%. The baselines lost clean utility mainly because the model unlocked or opened the door in clean stale/replay tasks even after a fresh "nobody home" reading. That behaviour is also unsafe (29.2% of clean unprotected runs). Six of the 24 clean tasks are completed by leaving the door locked, so blocking door access costs nothing there. The utility risk lies in the three tasks where unlocking is correct: the occupied unlock and the two arrivals.

Attack families and lockout cost. Table 2 separates the families. For the unprotected agent, hidden instructions led to unsafe effects at a task-level rate of one half, and stale or replayed evidence at three quarters. EffectShield prevented every unsafe effect in both families. All of its 3/56 attack successes are lockouts in the two occupied arrival tasks. After a replayed or stale "absent" reading, the agent did not read presence again, and the door stayed locked. Counted only where the matched clean run completed, the figure is the same. This is the expected cost of rules R4 and R5: the mediator refuses evidence it cannot trust but cannot make the agent fetch better evidence.

Decisions against the independent label. Of the mediator's 133 effect decisions, it blocked 40 policy-unsafe proposals and allowed 0; it blocked 2 permissible proposals and allowed 86; and it repaired 5 unsafe setpoints to permissible ones. That gives a recall of 100.0%, a block precision of 95.2% and a false-block rate of 2.3%. Repairs form a third disposition, graded by their executed outcome rather than counted as true positives. The weighted loss, ten times the unsafe executed actions plus the unnecessary blocks, was 2 for EffectShield against 570 unprotected and 480 with the safety prompt.

Ablations. Replaying each valid recorded proposal sequence through ablated mediators attributes the protection on stale and replayed evidence to R4–R5. Without them, 12 replays of unprotected proposals become unsafe (Table 3). Removing provenance (R3) mattered little,

Table 3: Trace-replay ablations: replays with an unsafe effect when the same recorded proposals are re-decided offline.

Source batch (valid runs)	Full mediator	Without R3 `no_provenance`	Without R4–R5 `no_freshness_replay`
Unprotected (111)	0	1	12
Safety prompt only (112)	0	0	12
EffectShield (112)	0	0	12

because device scope (R2) still stops most injected out-of-scope effects. These are fixed-trace results: the proposals do not adapt to blocks.

Cost and latency. One mediator decision took a median of 82 μs (p95 112 μs, $n = 695$; offline replay on an Apple M1 Pro). Median end-to-end task latency was 3.59 s for EffectShield and 3.76 s unprotected, so mediation accounts for well under 0.01% of task time. All 512 live attempts of the project used 946 generation calls, at an estimated USD 0.2255 by the published tariff (USD 0.1829 for the final study). The 1041 token-count preflight requests are assumed free. Estimates are not yet reconciled with the provider's invoice.

7 Discussion and limitations

What the results support. Within the declared scope, deterministic mediation of effects removed every unsafe effect that one capable model produced under hidden instructions and stale or replayed evidence, without reducing clean utility. The safety prompt reduced attacked unsafe effects only partly, which is consistent with the view that model-level instructions cannot guarantee enforcement [3, 18].

Threats to validity.

- *Construct.* The policy label and several grader predicates encode the same safety rules the mediator enforces, for example presence before unlocking, so they test faithful enforcement of a specification rather than the specification's adequacy.
- *Internal.* The same team designed the rules, the corpus and the grader. Splits, a held-out pilot, frozen goals and a single frozen run limit, but do not remove, alignment between rule design and test tasks.
- *Statistical.* There are 24 tasks and, at temperature 0, repetitions add little independent information; the task-cluster bootstrap reflects this. The outage removed 22% of attempts. Its timing was independent of condition, and the sensitivity analysis bounds its effect.
- *External.* The study uses one model, one synthetic five-device simulator, three authored attack families and a scripted continuation protocol. Real homes involve more devices, noisy sensors, other attackers and other models.

Limitations of the design. Freshness and replay checks cover presence evidence for door access only. Repair is limited to two templates. The lockout cost shows that refusing untrusted evidence must be paired with agent behaviour that re-reads the sensor; a mediator that prompts or performs the read itself is a direction for future work.

8 Reproducibility and artifact

All 432 attempts, including invalid ones, are retained with hashed traces, grades, model messages, decisions and costs. The freeze binds the protocol, manifest, conditions and implementation (`f82c4012`, reproduced by commit `1ec6544`). The commands `verify-freeze`, `verify-comparison`, `analyze` and `ablate` regenerate every reported number without credentials. In a fresh Python environment without third-party packages, the regenerated analysis and ablations were byte-identical. An artifact manifest checksums every evidence item. A read-only Streamlit dashboard presents the results with denominators and intervals, and shows untrusted text only as plain data. Model regeneration is not reproducible: the provider does not support seeds.

9 Conclusion

EffectShield shows that a small set of deterministic, evidence-aware rules between an agent and its actuators can eliminate the unsafe effects of injected instructions and stale or replayed sensor evidence in a controlled smart-home setting, at negligible latency and without loss of clean utility. The residual cost is lockouts, which arise when an agent does not refresh evidence after a block, and that cost points to the next design problem.

Contributions and AI assistance. Simon led the scenarios, attacks, independent grading, experiments, analysis and this report. Ahmed led the runtime interfaces, simulator, gateway, agent integration and the mediator, and independently replayed the final results. Both reviewed the protocol, the security-sensitive changes and the interpretation of results. Parts of the implementation, analysis and writing were produced with AI assistance (Claude Code); the project's prompt history and development log record that assistance.

References

[1] J. P. Anderson. *Computer Security Technology Planning Study.* ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972.

[2] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024.

[3] E. Debenedetti et al. Defeating prompt injections by design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pp. 587–618, 2026. doi:10.1109/SaTML68715.2026.00040.

[4] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang. Shield synthesis: Runtime enforcement for reactive systems. In *TACAS*, LNCS 9035, pp. 533–548, 2015. doi:10.1007/978-3-662-46681-0_51.

[5] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[6] M. Costa et al. Securing AI agents with information-flow control. arXiv:2505.23643, 2025.

[7] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. Preserving statistical validity in adaptive data analysis. In *STOC*, 2015. arXiv:1411.2664.

[8] B. Efron. Bootstrap methods: Another look at the jackknife. *Annals of Statistics*, 7(1):1–26, 1979.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90, 2023. arXiv:2302.12173.

[10] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449, 2023.

[11] B. Nassi, S. Cohen, and O. Yair. Invitation is all you need! Promptware attacks against LLM-powered assistants in production are practical and dangerous. arXiv:2508.12175, 2025.

[12] Y. Ruan et al. Identifying the risks of LM agents with an LM-emulated sandbox. In *ICLR*, 2024.

[13] V. Saravanan, G. J. Berman, and S. J. Sober. Application of the hierarchical bootstrap to multi-level data in neuroscience. *Neurons, Behavior, Data Analysis, and Theory*, 2020. arXiv:2007.07797.

[14] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[15] G. Seo et al. SimuHome: A temporal- and environment-aware benchmark for smart home LLM agents. In *ICLR*, 2026.

[16] T. Shi et al. Progent: Programmable privilege control for LLM agents. arXiv:2504.11703, 2025.

[17] H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *ICSE*, pp. 2938–2950, 2026. doi:10.1145/3744916.3764546.

[18] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *NDSS*, 2025. doi:10.14722/ndss.2025.241131.

[19] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of ACL*, 2024.

[20] H. Zhang et al. PromptShield Home: Ambient multimodal prompt injection defense for smart-home agents. UbiComp 2026 poster; arXiv:2608.05495.